

Foundations of Artificial Intelligence

Embedding and Manifold Learning

Andrea Torsello

Multidimensional Scaling (MDS)

- It is often easier to provide distances
- What can we do if we only have distances
- **Multidimensional scaling**
 - Compute the low dimensional representation $\varphi \in \mathbb{R}^q$ that most faithfully preserves pairwise distances of data $\mathbf{x} \in \mathbb{R}^m$ (or similarities which is inversely proportional to distances)
- Euclidean distance (squared) between two points

$$d_{ij}^2 = (x_i - x_j)^\top (x_i - x_j) = x_i^\top x_i + x_j^\top x_j - 2x_i^\top x_j$$

- Assume, without loss of generality, that the centroid of the configuration of n points is at the origin

$$\sum_{i=1}^n \mathbf{x}_{ij} = 0, \quad j = 1, \dots, m$$

- We have

$$\begin{aligned} \frac{1}{n} \sum_{j=1}^n d_{ij}^2 &= \mathbf{x}_i^\top \mathbf{x}_i + \frac{1}{n} \sum_{j=1}^n \mathbf{x}_j^\top \mathbf{x}_j \\ \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n d_{ij}^2 &= \frac{2}{n} \sum_{i=1}^n \mathbf{x}_i^\top \mathbf{x}_i \end{aligned}$$

- We can define the $n \times n$ *Gram matrix* G of inner products from the matrix of squared euclidean distances

$$\begin{aligned}
 G_{ij} &= x_i^\top x_j \\
 &= -\frac{1}{2} \left(d_{ij}^2 - \frac{1}{n} \sum_{j=1}^n d_{ij}^2 - \frac{1}{n} \sum_{i=1}^n d_{ij}^2 + \frac{1}{n^2} \sum_{i=1}^n \sum_{j=1}^n d_{ij}^2 \right)
 \end{aligned}$$

- This process is called *centering*
 - Define matrix A as $a_{ij}=d_{ij}^2$, the Gram matrix G is

$$G = HAH$$

where H is the centering matrix,

$$H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$$

- If d is a squared Euclidean distance, the Gram matrix is positive definite (linear kernel)
- G can be written in terms of spectral decomposition

$$XX^T = G = U\Sigma U^T = (U\Sigma^{1/2})(U\Sigma^{1/2})^T = \Phi\Phi^T$$

- Thus $X = \Phi$ modulo isometries.
- We can have an exact Euclidean embedding of the data if and only if the Gram matrix is positive semidefinite.
- In general, use only the k largest positive eigenvectors
 - Minimize

$$\mathcal{E}_{MDS} = \sum_i \sum_j (\mathbf{x}_i \cdot \mathbf{x}_j - \phi_i \cdot \phi_j)^2$$

- What can we do if the data cannot be embedded in a Euclidean space?
 - The gram matrix has negative eigenvalues

1. Ignore: take only positive eigenvalues

2. Correct: change the distances so that eigenvalues are all positive

- We can add a constant to all distances, In fact if G is a Gram matrix

then the matrix $G' = G - \lambda_n(G)I$ is positive semidefinite and is the Gram

matrix of the squared distance matrix $D^{2'} = D^2 - \lambda_n(G)(\mathbf{1}\mathbf{1}^T - \mathbf{I})$

3. Other...

Relations to learning algorithms

- Embedding transformations are related to properties of learning algorithms
- E.g., consider k-means clustering algorithm
 - Since the centroids are linear with respect to the points' positions, we can express the distance to the centroids in terms of the distances between the points

$$||x_i - \mu_k||^2 = \frac{1}{N_k} \sum_{j=1}^n r_{jk} ||x_i - x_j||^2 - \frac{1}{2N_k^2} \sum_{j=1}^n \sum_{l=1}^n r_{jk} r_{lk} ||x_j - x_l||^2$$

- or

$$\sum_{i=1}^n r_{ik} ||x_i - \mu_k||^2 = \frac{1}{2N_k} \sum_{j=1}^n \sum_{l=1}^n r_{jk} r_{lk} ||x_j - x_l||^2$$

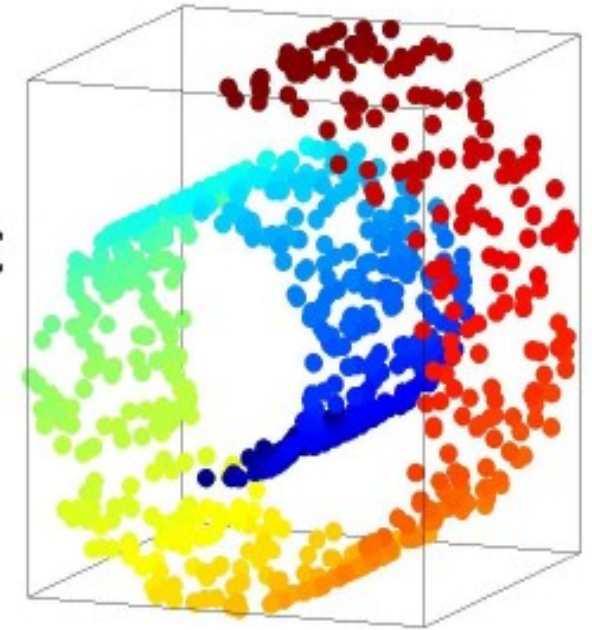
- Thus, its energy function can be expressed in terms of pairwise distance data

$$J = \frac{1}{2} \sum_{k=1}^K \frac{\frac{1}{2N_k} \sum_{j=1}^n \sum_{l=1}^n r_{jk} r_{lk} ||x_j - x_l||^2}{\sum_{j=1}^n r_{jk}}$$

- It is easy to see that a constant shift in the distances does not affect the minimizers of the distortion J
- To cluster data expressed in terms of non-euclidean data we can perform a shift embedding without affecting the cluster structure.

Manifold learning

- Find a low-D basis for describing high-D data.
- uncovers the intrinsic dimensionality (invertible)
- Why?
 - data compression
 - curse of dimensionality
 - de-noising
 - visualization
 - reasonable distance metrics



Example

- appearance variation



Braeburn



Cortland



Fuji



Gala



Ginger Gold



Golden Delicious



Red Delicious



Granny Smith



Honeycrisp



Jonathan



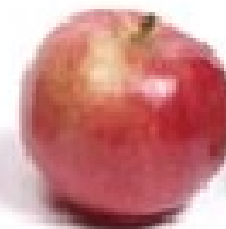
Jonagold



McIntosh



Pacific Rose



Paula Red



Wealthy

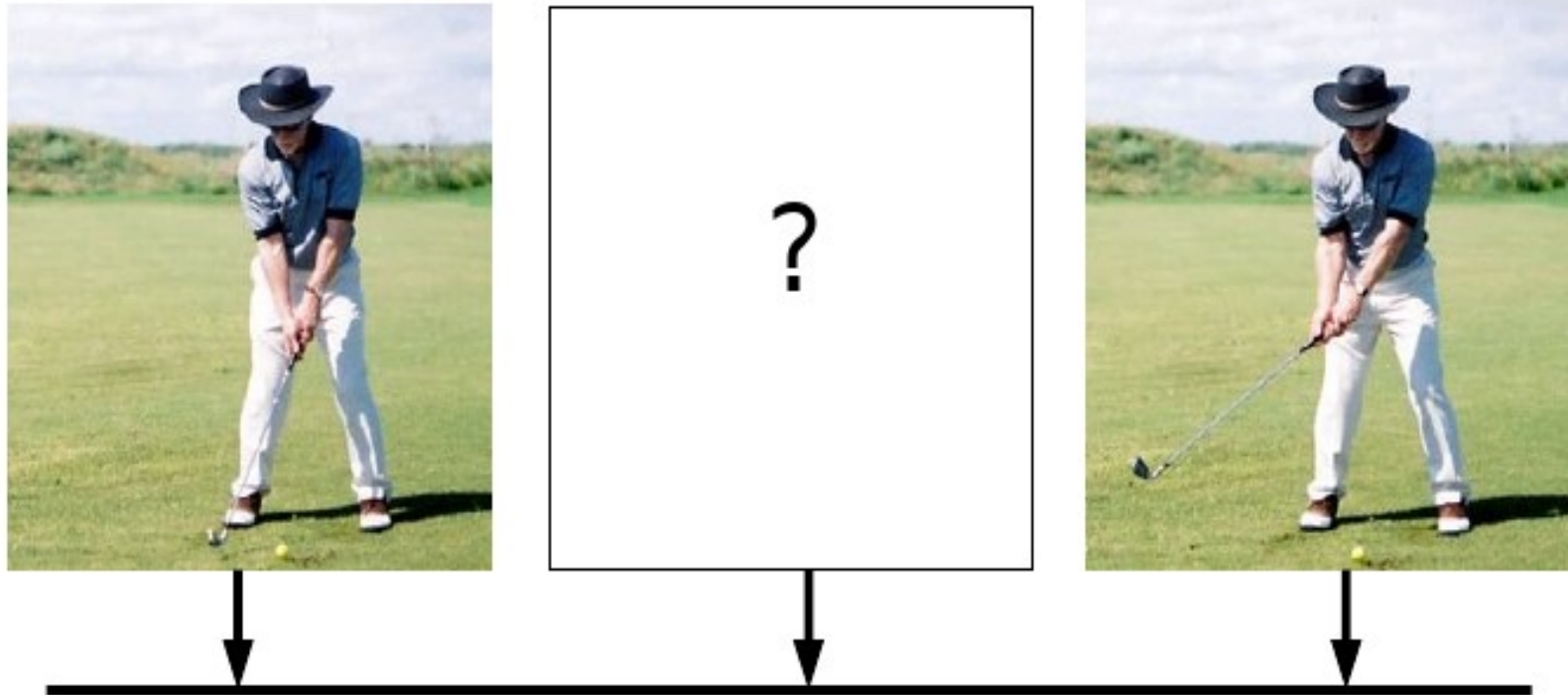
Example

- Deformation



Reasonable distance metrics

- What is the mean position in the timeline?





Linear interpolation

Manifold interpolation



Kernel PCA

- PCA cannot handle non-linear manifolds
- We have seen that the “Kernel trick” can “bend” linear classifiers
- Can we use the kernel trick with PCA?

- Assume that the data already has mean 0

- The Covariance matrix is

$$S = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T$$

- Not a dot product!

- Assume a non-linear map ϕ onto an M-dimensional space that maps $\mathbf{x}$ onto barycentric coordinates

$$\sum_{i=1}^n \phi(\mathbf{x}_i) = 0$$

- In that space the sample covariance is

$$C = \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T$$

- Assume C has eigenvectors

$$C \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

- We have

$$\frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) (\phi(\mathbf{x}_i)^T \mathbf{v}_j) = \lambda_j \mathbf{v}_j$$

from which we conclude that $\mathbf{v}$ can be expressed as a linear combination of $\phi(\mathbf{x}_i)$

$$\mathbf{v}_i = \sum_{j=1}^n a_{ij} \phi(\mathbf{x}_j)$$

- Substituting we have:

$$\frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T \sum_{k=1}^n a_{jk} \phi(\mathbf{x}_k) = \lambda_j \sum_{m=1}^n a_{jm} \phi(\mathbf{x}_m)$$

- Left-multiplying by $\phi(\mathbf{x}_l)$ and setting $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$, we have

$$\frac{1}{n} \sum_{i=1}^n k(\mathbf{x}_l, \mathbf{x}_i) \sum_{k=1}^n a_{jk} k(\mathbf{x}_i, \mathbf{x}_k) = \lambda_j \sum_{i=1}^n a_{ji} k(\mathbf{x}_l, \mathbf{x}_i)$$

or, in matrix notation

$$\mathbf{K}^2 \mathbf{a}_i = \lambda_i N \mathbf{K} \mathbf{a}_i$$

- Solving the eigenvalue equation

$$\mathbf{K}\mathbf{a}_i = \lambda_i N \mathbf{a}_i$$

you find the coefficients $\mathbf{a}_i$

- The normalization condition is obtained from the normality of the $\mathbf{v}_i$:

$$1 = \mathbf{v}_k^T \mathbf{v}_k = \sum_{i=1}^n \sum_{j=1}^n a_{ki} a_{kj} \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \mathbf{a}_k^T \mathbf{K} \mathbf{a}_k = \lambda_k N \mathbf{a}_k^T \mathbf{a}_k$$

- The projections onto the principal components are

$$y_i(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{v}_i = \sum_{j=1}^n a_{ij} \phi(\mathbf{x})^T \phi(\mathbf{x}_j) = \sum_{j=1}^n a_{ij} k(\mathbf{x}, \mathbf{x}_j)$$

- We have assumed that ϕ maps $\mathbf{x}$ onto barycentric coordinates
 - We want to generalize it

- Let ϕ be any map and

$$\tilde{\phi}(\mathbf{x}) = \phi(\mathbf{x}) - \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i)$$

a corresponding *centralized* map.

- The corresponding centralized kernel will be

$$\begin{aligned} \tilde{K}_{ij} &= \tilde{\phi}(\mathbf{x}_i)^T \tilde{\phi}(\mathbf{x}_j) = \\ &= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) - \frac{1}{n} \sum_{l=1}^n \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_l) - \frac{1}{n} \sum_{l=1}^n \phi(\mathbf{x}_l)^T \phi(\mathbf{x}_i) + \frac{1}{n^2} \sum_{l=1}^n \sum_{m=1}^n \phi(\mathbf{x}_l)^T \phi(\mathbf{x}_m) = \\ &= k(\mathbf{x}_i, \mathbf{x}_j) - \frac{1}{n} \sum_{l=1}^n k(\mathbf{x}_i, \mathbf{x}_l) - \frac{1}{n} \sum_{l=1}^n k(\mathbf{x}_l, \mathbf{x}_i) + \frac{1}{n^2} \sum_{l=1}^n \sum_{m=1}^n k(\mathbf{x}_l, \mathbf{x}_m) \end{aligned}$$

or, in matrix notation

$$\tilde{\mathbf{K}} = \mathbf{K} - \frac{1}{n} \mathbf{1}^T \mathbf{K} - \frac{1}{n} \mathbf{K} \mathbf{1} + \frac{1}{n^2} \mathbf{1}^T \mathbf{K} \mathbf{1}$$

Examples

- Gaussian kernel

$$k(\mathbf{x}, \mathbf{x}') = e^{-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{0.1}}$$

Figure 10.1: $\sigma = 2^{-1/2}$

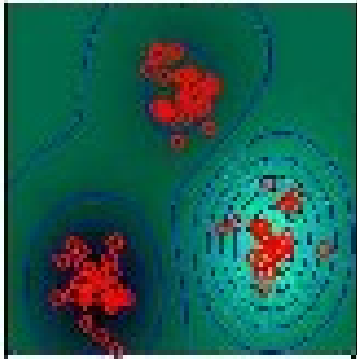


Figure 10.2: $\sigma = 2^{-1/2}$

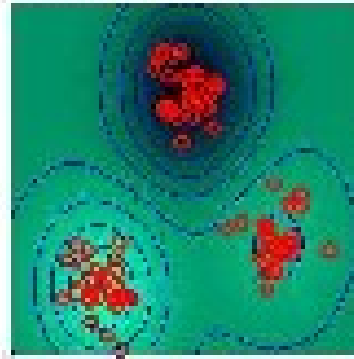


Figure 10.3: $\sigma = 2^{-1/2}$

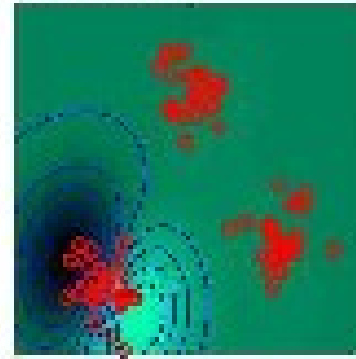


Figure 10.4: $\sigma = 2^{-1/2}$

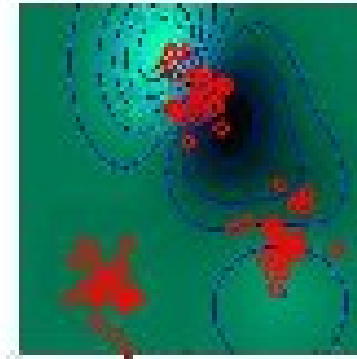


Figure 10.5: $\sigma = 2^{-1/2}$

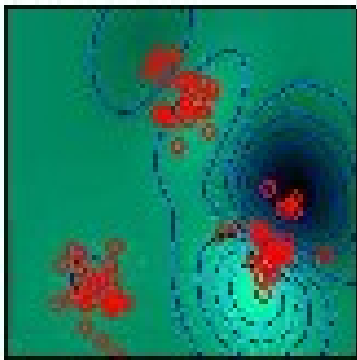


Figure 10.6: $\sigma = 2^{-1/2}$

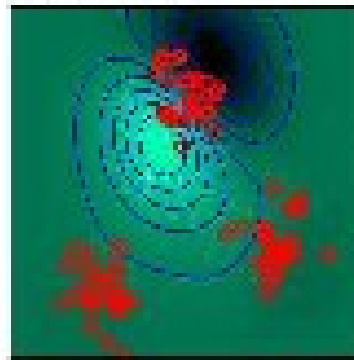


Figure 10.7: $\sigma = 2^{-1/2}$

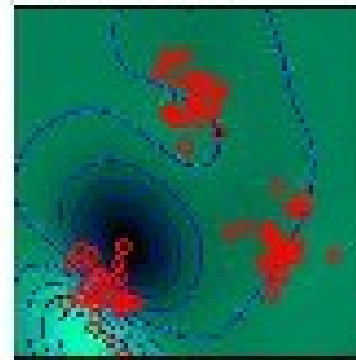
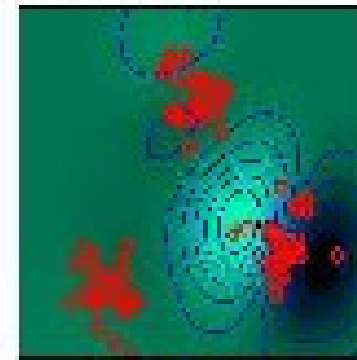


Figure 10.8: $\sigma = 2^{-1/2}$



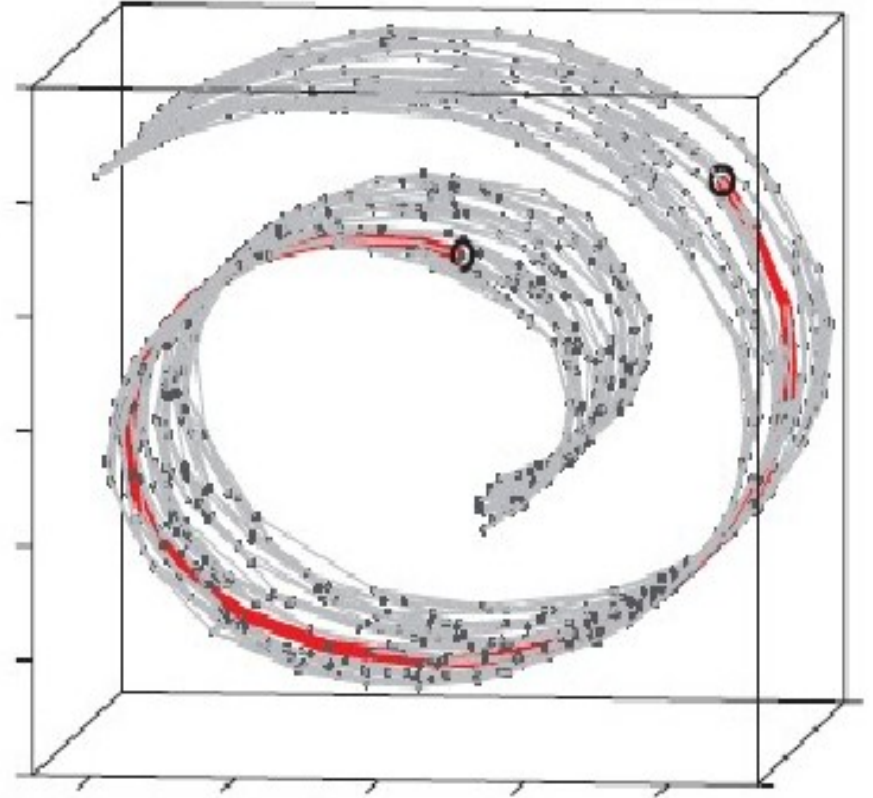
Problems with kernel PCA

- Kernel PCA uses the kernel trick to find latent coordinates of a non-linear manifold
- However:
- It works on the $n \times n$ matrix $\tilde{\mathbf{K}}$ rather than the $D \times D$ covariance matrix
 - Problem for large datasets
 - On the other hand, non linear generalization requires large amount of data
- Can map from data to the latent space, but not *viceversa*:
 - A linear combination of elements on a non-linear manifold might not lie on the manifold

Isomap

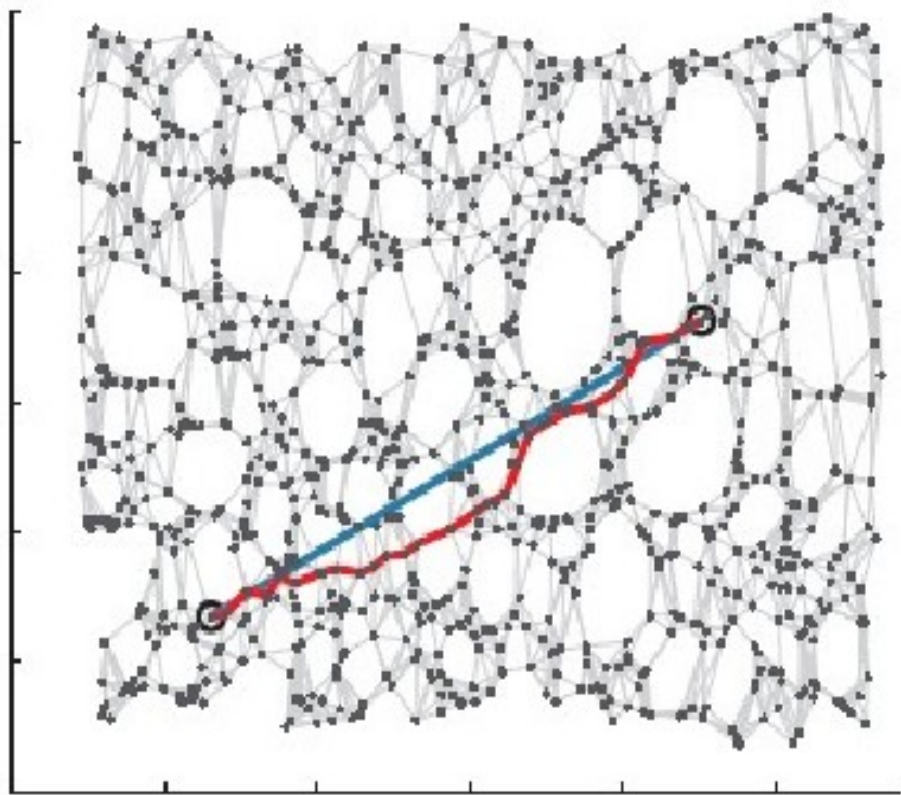
- 2. Build a sparse graph with K-nearest neighbors

$$D_g = \left[\text{blue ellipse} \right]$$



- 2. Infer other interpoint distances by finding shortest paths on the graph (Dijkstra's algorithm).

- 3. Build a low-D embedded space to best preserve the complete distance matrix.



Error function:

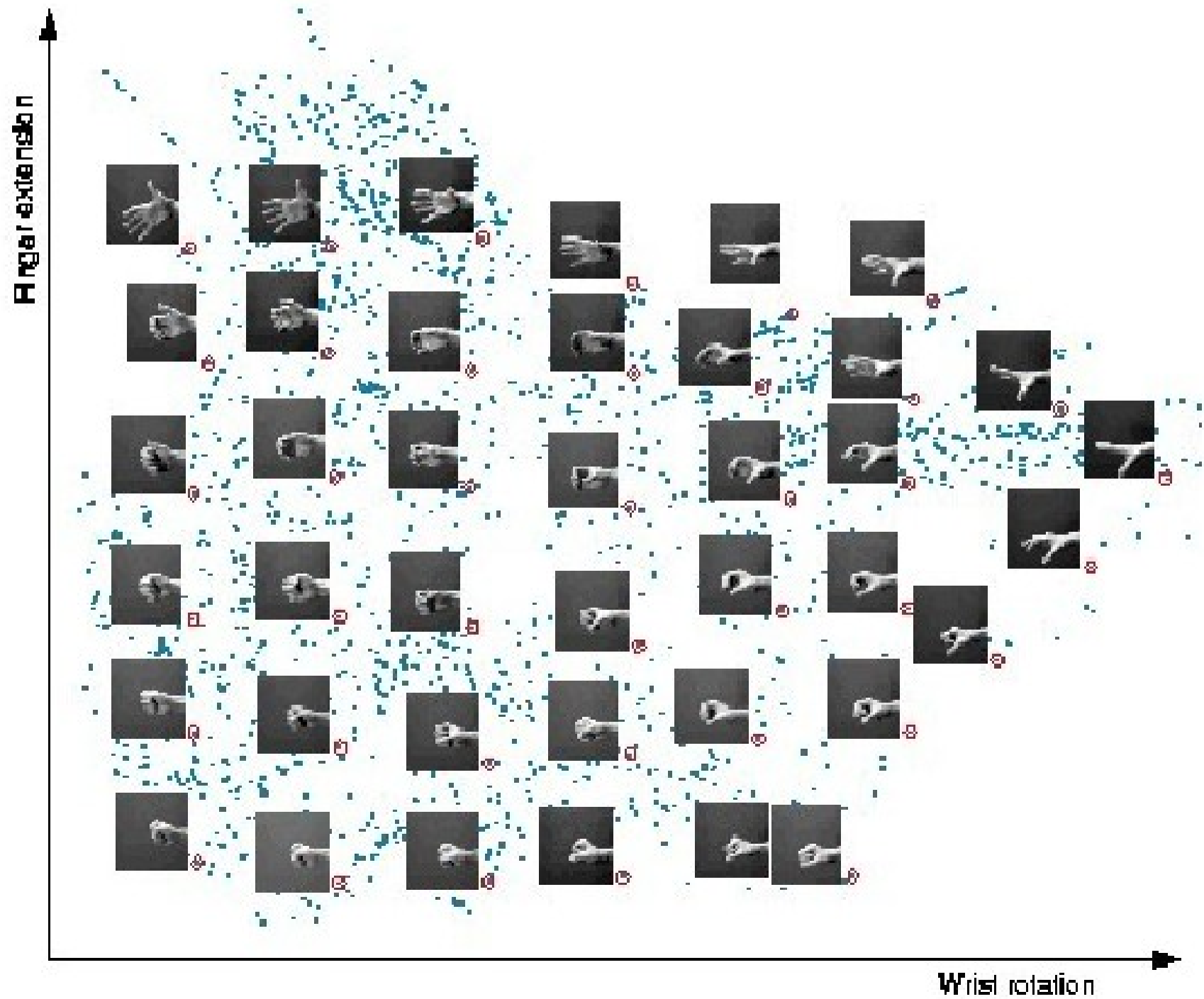
inner product
distances in
graph

inner product
distances in new
coordinate
system

MDS

L2 norm

$$E = \|\tau(D_G) - \tau(D_Y)\|_{L^2}$$



Laplacian Eigenmap

-
-

Step 1. [*Constructing the Graph*] We put an edge between nodes i and j if x_i and x_j are “close”. There are two variations:

- (a) ϵ -neighborhoods. [*parameter* $\epsilon \in \mathbb{R}$] Nodes i and j are connected by an edge if $\|\mathbf{x}_i - \mathbf{x}_j\|^2 < \epsilon$.

Advantages: geometrically motivated, the relationship is naturally symmetric.

Disadvantages: often leads to graphs with several connected components, difficult to choose ϵ .

- (b) n nearest neighbors. [*parameter* $n \in \mathbb{N}$] Nodes i and j are connected by an edge if i is among n nearest neighbors of j or j is among n nearest neighbors of i .

Advantages: simpler to choose, tends to lead to connected graphs.

Disadvantages: less geometrically intuitive.

Step 2. [Choosing the weights] Here as well we have two variations for weighting the edges:

- (a) Heat kernel. [*parameter* $t \in \mathbb{R}$]. If nodes i and j are connected, put

$$W_{ij} = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{t}}$$

The justification for this choice of weights will be provided later.

- (b) Simple-minded. [*No parameters*]. $W_{ij} = 1$ if and only if vertices i and j are connected by an edge.

A simplification which avoids the necessity of choosing t .

- Search for an embedding that penalizes large distances between strongly connected nodes

$$\text{minimize} \quad \sum_{i,j} (y_i - y_j)^2 W_{ij}$$

$$\text{subject to} \quad \sum_i y_i^2 D_{ii} = 1$$

$$D_{ii} = \sum_j W_{ij}$$

Where D is the usual degree matrix

- the constraint $\mathbf{y}^T \mathbf{D} \mathbf{y} = 1$ removes the arbitrary scaling factor in the embedding.
- Note that the optimization problem can be cast in term of the generalized eigenvector problem

$$L \mathbf{y} = \lambda D \mathbf{y}$$

where $L = D - W$ is the Laplacian, since

$$\sum_{i,j} (y_i^2 + y_j^2 - 2y_i y_j) W_{ij} = \sum_i y_i^2 D_{ii} + \sum_j y_j^2 D_{jj} - 2 \sum_{i,j} y_i y_j W_{ij} = 2 \mathbf{y}^T L \mathbf{y}$$

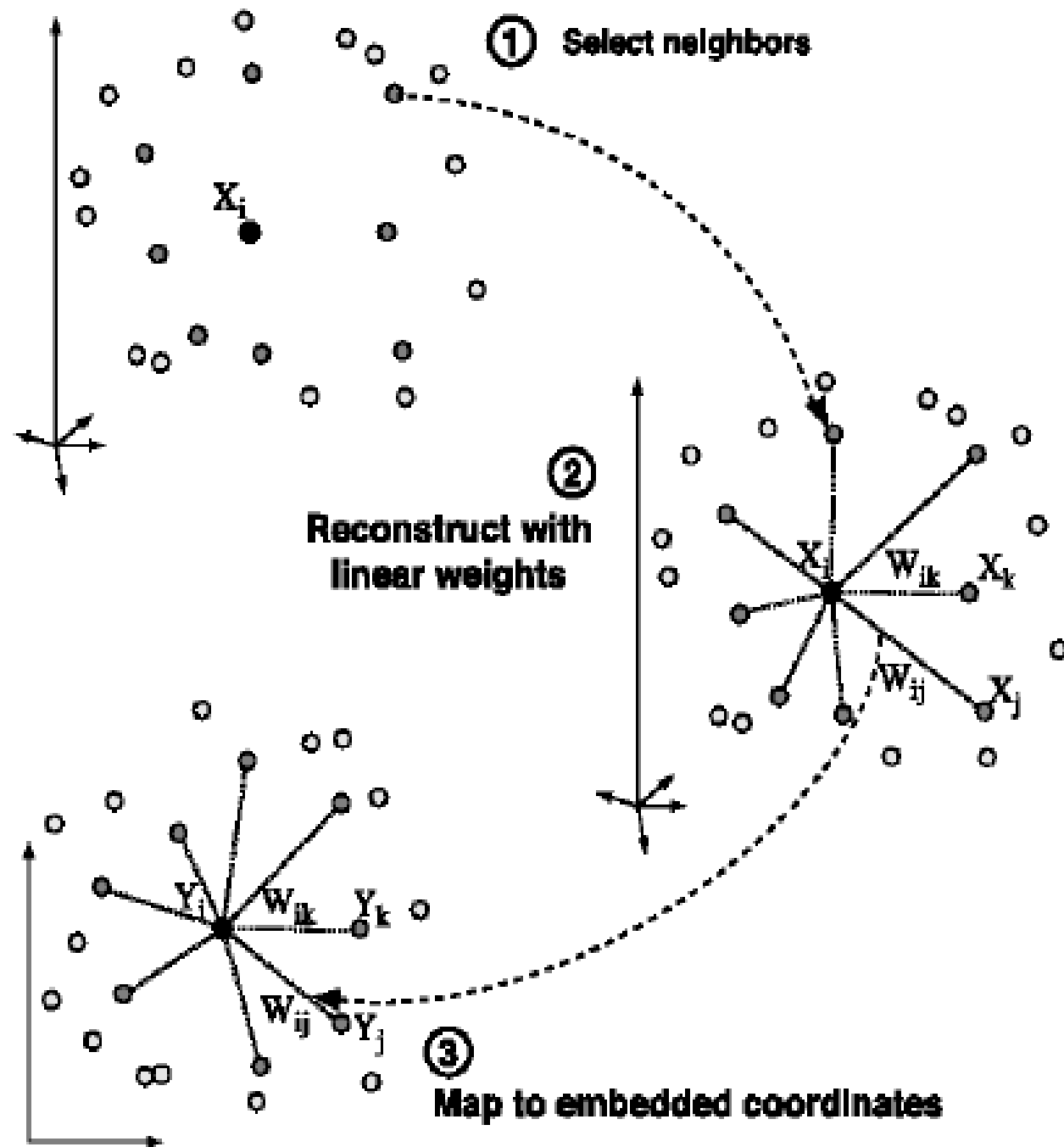
Locally Linear Embedding (LLE)

- Find a mapping to preserve local linear relationships between neighbors

1. Compute the neighbors of each data point, $\vec{X}_i$.

2. Compute the weights W_{ij} that best reconstruct each data point $\vec{X}_i$ from its neighbors, minimizing the cost in Equation (1) by constrained linear fits.

3. Compute the vectors $\vec{Y}_i$ best reconstructed by the weights W_{ij} , minimizing the quadratic form in Equation (2) by its bottom nonzero eigenvectors.



Compute weights

- For each data point i in D dims, we find the K nearest neighbors
- Compute a kind of local principal component plane to the points in the neighborhood, minimizing

$$\sum_i \left| x_i - \sum_j W_{ij} x_j \right|^2$$

over weights W_{ij} satisfying

- W_{ij} is the contribution of point j to the reconstruction of point i .

- Least square solution obtained by solving

$$\Delta_i' \Delta_i w_i = \mathbf{1}$$

where

$$\Delta_i = [x_i - x_{j_1}, \dots, x_i - x_{j_{r(i)}}]$$

- weight-based representation has several desirable invariances
 - It is invariant to any local rotation or scaling of x_i and its neighbors
 - on or scaling of x_i and its neighbors (due to the linear relationship
 - the normalization requirement on W_{ij} adds invariance to translation

$$\sum_j W_{ij} (x_j + \alpha) = x_i + \sum_j W_{ij} \alpha = x_i + \alpha$$

- The mapping preserves angle and scale within each local neighborhood.

- Having solved for the optimal weights which capture the local structure, find new locations which approximate those relationships.
- This can be done minimizing the same quadratic cost function for the new data locations:

$$\tilde{Y} = \arg \min_Y \sum_{i=1}^n \left\| y_i - \sum_{j \in \Gamma(i)} \tilde{W}_{ij} y_j \right\|^2$$

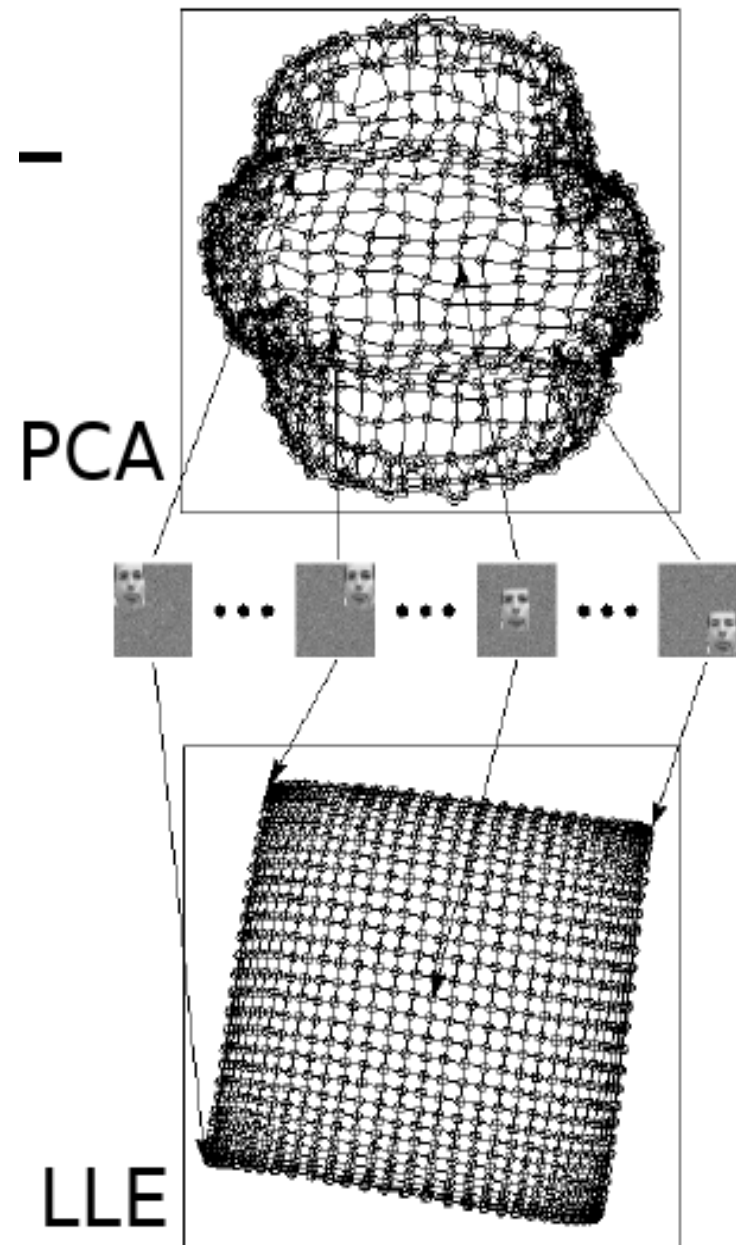
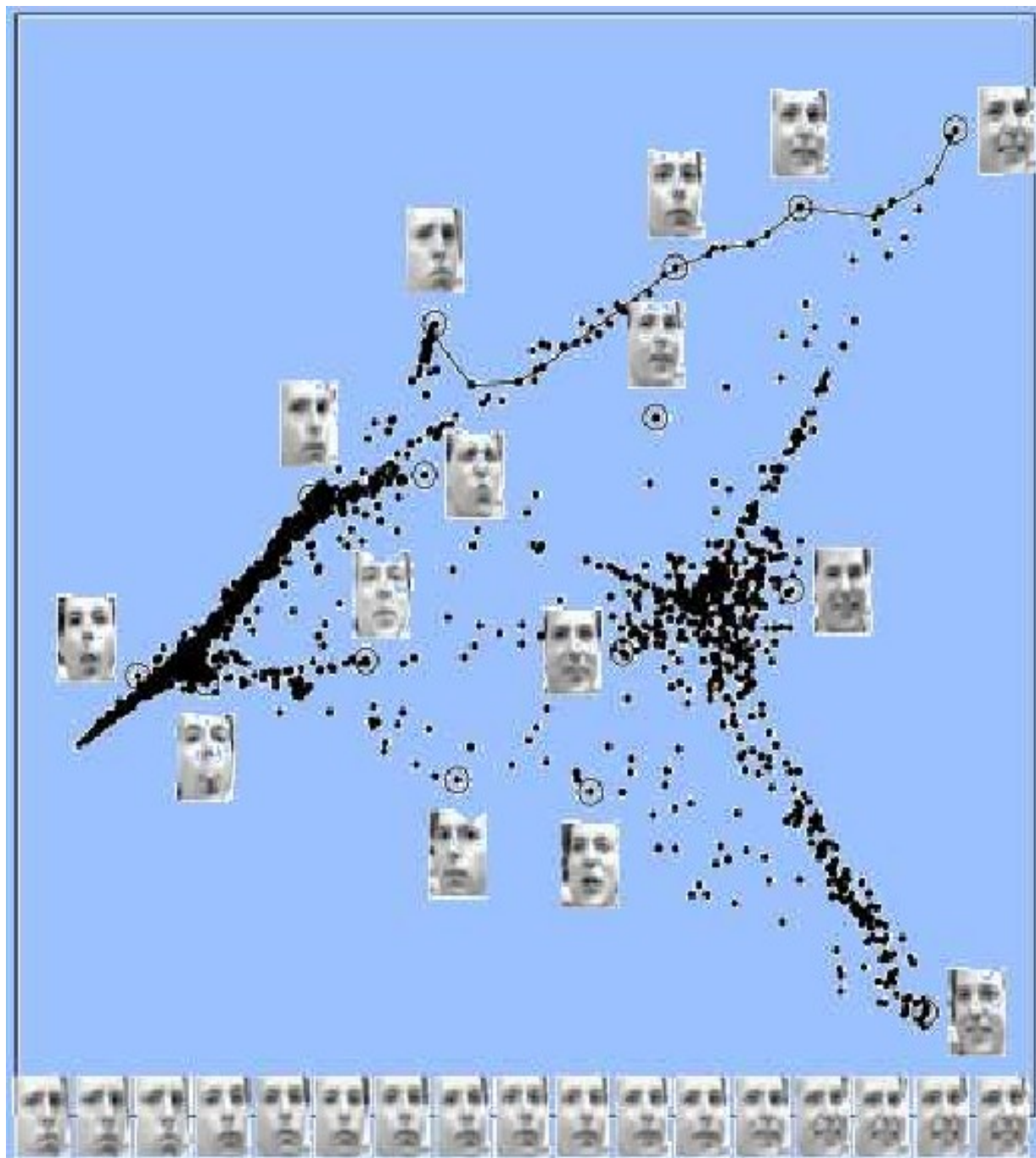
- In order to avoid spurious solution we add the constraints

$$Y\mathbf{1} = \mathbf{0}, \quad YY' = I_n$$

- We can rewrite the problem as

$$\begin{aligned} &\text{minimize} \quad \text{trace} \left((Y - WY)^T (Y - WY) \right) = \text{trace} \left(Y^T (I - W)^T (I - W) Y \right) \\ &\text{subject to} \quad \mathbf{1}^T Y = 0, \quad \frac{1}{n} Y^T Y = I \end{aligned}$$

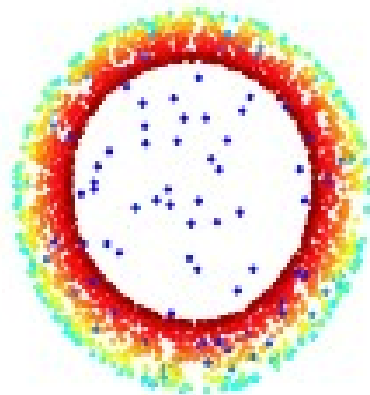
- Solved by the d smallest eigenvectors
 - $\mathbf{1}^T Y = \mathbf{0}$ implies that you must skip the smallest.



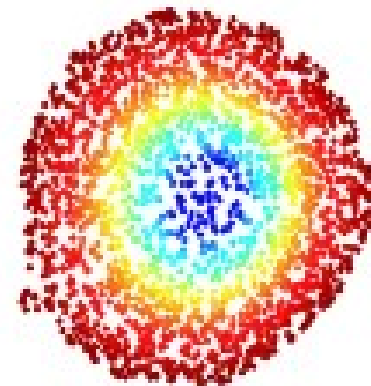
- Isomap: pro and con
 - preserves global structure
 - long-range distances become more important than local structure
 - few free parameters
 - sensitive to noise, noise edges
 - computationally expensive (dense matrix eigen-reduction)
- LLE: pro and con
 - no local minima, one free parameter
 - incremental & fast
 - simple linear algebra operations
 - can distort global structure
 - Preserves local structure over long-range distances



data



isomap



LLE

- No matter what your approach is, the “curvier” your manifold, the denser your data must be

